

Ten Years of SAP, and What Odoo Taught Me in a Week

A short account of the journey behind this pitch — and the dozens of passes it actually took — for whoever's curious after the meeting

Waqas Abbas Mian · Senior ERP Solution Architect

I want to leave you with something shorter and more personal than the package itself — not another proof point, but the honest account of how I got here, including the part that doesn't show up in a polished PDF: how many times this was wrong before it was right, and whose judgement caught it each time.

What the iterations actually looked like

This package did not arrive in one pass. It went through dozens of rounds — a live build, then an audit of that build, then a peer review of the audit, then a live regression test of the peer review's own findings, each round stricter than the last — because I directed it that way, not because a tool volunteered the rigour. I used AI tooling throughout, and I want to be specific about the shape of that collaboration rather than wave at it: the tool moved fast and held detail I couldn't hold in my head across fifteen cross-referenced documents and a live system at once; I supplied the standard it had to clear, and I was the one who kept sending work back. A few of those redirects are worth naming, because they show the direction travelling one way and the platform knowledge travelling the other: when the first screenshots of the demo came back looking like generic SaaS rather than Odoo, I sent them back to study the actual product before touching the render again. When a written review of my own work claimed a set of "critical defects," I didn't accept the write-up — I had it log back into the live system and verify every claim against reality, which found that some were real and had to be fixed, and others were simply false. When the package's language didn't match a German client and a UK hiring panel, I corrected the convention rather than let a plausible-sounding default stand. None of that judgement came from the tool. It came from a decade of sitting in the room when a client asks the question a document didn't anticipate, and knowing which answer survives that room.

Where the discipline came from

I never studied computer science. I built my way into enterprise software from the ground up — running a Shell forecourt through college, running a warehouse's WMS and its returns queue by hand, teaching myself SQL to keep an e-commerce site alive — and then ten years inside the SAP ecosystem: C4/HANA, Hybris, Cloud for Customer, Sales and Service Cloud, most recently CRM architecture at enterprise scale. What that decade actually taught me wasn't a product. It was a **lifecycle** — blueprint, workshop, functional spec, build, UAT, hypercare — and a rule that survives any platform: nothing you show a client is real until you've tried to break it yourself first. That rule is the reason this package went through as many rounds as it did.

What this case study actually taught me about Odoo

I had never configured Odoo before this brief. What I found, learning it under deadline, surprised me: it isn't a lesser SAP — it's a different bet entirely. SAP separates and specialises; Odoo unifies one data model and lets automation rules carry the compliance logic that SAP would spread across modules and middleware. For a mid-market regulated manufacturer, that unification is the whole value proposition — I could direct the build of a real GxP-aware compliant thread, four enforced controls, and full UDI traceability in days, on a platform that costs a fraction of the enterprise suites I spent a decade implementing. That's not a knock on SAP; it's a precise answer to *which client needs which tool*, which is the question a decade of SAP work actually trains you to ask — and the knowledge transfer ran both ways here: the tooling taught me Odoo's mechanics fast enough to direct a real build under deadline; my decade of delivery taught it what "done" actually means before a client sees it.

What I learned about much.

I read your MedTech page closely enough to notice what it doesn't yet say about the UK's 2025–26 device reforms, and I found your Managing Partner's own conference talk before I ever met anyone from your team — not to flatter it back to you, but because understanding a firm's actual published thinking is the same discovery discipline I'd bring to a client engagement. What came through in that research, and in the case brief itself, is a business that would rather say "here's what we don't know yet" than oversell a capability — which is exactly the instinct this whole package was built around, redirect by redirect. That's not a coincidence I'm pointing out to flatter you; it's the reason I think I'd fit here without having to unlearn anything.

THE HONEST SUMMARY

I didn't come into this case study as an Odoo expert, and I'm not going to pretend otherwise. I came in as someone who has spent a decade learning that the platform is never really the hard part — the discipline is, and the discipline is what you apply round after round until the work survives contact with someone trying to break it. That's what these weeks actually were: not one clever build, but a long argument between what a tool produced and what my experience would accept — repeated until they agreed. That's what I'd bring to your team on day one, on whichever platform your next client happens to be running.

The full journey, start to finish

- 1 Live build**
A working Odoo Enterprise instance configured over the API — credential gate, quarantine hold, UDI traceability, consignment, instrument reprocessing — before a single document was drafted.
- 2 Multiple builds, one verified**
Several independent AI-driven attempts were compared and audited against each other; the defects each one produced — a contradicted hold date, an empty consignment, an invisible field — were caught and fixed before any of it was trusted.
- 3 Delivery-grade package**
Fifteen client documents built in one consistent house style, cross-referenced end to end — brief → requirement → mechanism → test → evidence.
- 4 Independent audit rounds**
Several full self-commissioned reviews, each stricter than the last, each one instructed to find fault rather than confirm quality.
- 5 Word-by-word peer review**
Every document read line by line for language, consistency and tone — including correcting the spelling convention once I confirmed the actual audience.
- 6 My own revision system**
A private preparation pack built alongside the client package: a Q&A drill, a technical deep-dive, a glossary, a rehearsal runbook — so nothing in the pitch depends on memory alone.
- 7 Researching much.**
Reading your published MedTech material and your Managing Partner's own conference talk closely enough to find a real, current gap — the UK's 2025–26 device reforms — and closing it as an unprompted addition.
- 8 Live regression & a hostile review of myself**
Every control re-tested against the running system, then a simulated boardroom teardown — a sceptical CIO, a QA lead, a CFO, a hostile reviewer — run on my own work before anyone else could run it on me.
- 9 Catching what a second AI session broke**
A separate tool touched the same live system while I wasn't watching it. I didn't take its own report at face value — I went back into the system myself, found what was real and what wasn't, and corrected the parts that were actually broken.
- 10 This note**
Written last, once everything above was already true — the honest account, not the highlight reel.

Ten stages, and most of them repeated more than once. That repetition is the actual answer to "how did you use AI" — not a single prompt, but a standard applied again and again until the work had nothing left to hide.